

FACG — Frequency Analysis of CPU and GPU mixed computing

A GPU-accelerated iterative prewhitening frequency analysis tool for astronomical time-series data, inspired by [SigSpec](#) (Reegen 2007, A&A 467, 1353).

FACG implements the full SigSpec pipeline — spectral significance estimation, iterative prewhitening, and global multi-sine optimisation — while leveraging CUDA GPUs for the most computationally intensive steps. When no GPU is available it falls back transparently to CPU (NumPy).

[Read this in Chinese \(中文版\)](#) 

Features

- **GPU acceleration** via CuPy — transparent fallback to NumPy on CPU
 - **NVIDIA**: via CuPy/CUDA
 - **Apple Silicon**: via PyTorch/Metal (MPS)
 - **SigSpec-compatible significance** formula (analytic false-alarm probability)
 - **Iterative prewhitening** with frequency refinement (bisection search)
 - **Global multi-sine optimisation** via Levenberg–Marquardt with analytic Jacobian
 - **Flexible file naming** — any legal filename accepted (no rigid naming rules)
 - **Smart hardware detection** — automatically detects GPU hardware and interactively prompts for missing dependencies (CuPy/PyTorch) before falling back to CPU
 - **Benchmark utility** — built-in `--testdata` generator for performance benchmarking
 - **Configuration file support** — generate a default `facg.conf` via `--gen-config` to save your preferred parameters
 - **Installable Python package** with CLI (`facg`) and Python API
 - **Processing time reporting** — per-iteration and total elapsed time
-

Package Structure

```

FACG/
├── pyproject.toml      # Package build configuration
├── README.md          # This document
├── FACG.py            # Original script (kept for reference)
├── facg/              # Installable Python package
│   ├── __init__.py    # Package entry, exports FACGConfig &
run_analysis
│   ├── __main__.py    # CLI entry point: facg / python -m facg
│   ├── backend.py     # GPU/CPU unified backend (CuPy ↔ NumPy)
│   └── config.py      # FACGConfig dataclass – all tuneable
parameters
│   └── spectrum.py    # Core spectral engine (DFT, significance,
refinement)
│       └── optimizer.py # Multi-sine global optimiser (GPU Jacobian
+ LM)

```

```

├── io.py                # File I/O (read any text file, write
results)
├── prewhiten.py         # Iterative prewhitening cascade (main
loop)

```

Module Responsibilities

Module	Role
<code>backend.py</code>	Auto-detects CuPy/CUDA; provides <code>xp</code> , <code>to_device()</code> , <code>to_host()</code> so all other modules are backend-agnostic
<code>spectrum.py</code>	<code>compute_significance_spectrum()</code> — GPU-accelerated batch DFT + SigSpec significance formula; <code>refine_frequency()</code> — bisection search on significance peak; <code>make_freq_grid()</code> — frequency grid construction
<code>optimizer.py</code>	<code>global_optimize()</code> — simultaneous Levenberg–Marquardt fit of all frequencies, amplitudes, and phases with GPU-computed analytic Jacobian
<code>prewhiten.py</code>	<code>run_analysis()</code> — the main cascade loop: detect → refine → optimise → subtract → repeat
<code>config.py</code>	<code>FACGConfig</code> dataclass holding all user-tuneable parameters with SigSpec-compatible defaults
<code>io.py</code>	Reads any whitespace-delimited text file; writes result / spectrum / residual / phase-diagram files with descriptive names
<code>__main__.py</code>	<code>argparse</code> -based CLI supporting multiple input files, all parameters as flags

Installation

```

cd FACG
pip install -e .

```

A single command installs everything. At runtime FACG **auto-detects** the computing backend:

- If a CUDA-capable GPU and CuPy are available → GPU mode (with a confirmation message).
- If on Apple Silicon and PyTorch is available → GPU mode via Metal.
- Otherwise → CPU mode, with a hint on how to enable GPU acceleration depending on your OS:

On **Windows/Linux** (for NVIDIA GPUs):

```

✓ CPU (NumPy 1.26.x)
△ NVIDIA GPU hardware detected, but CuPy is not installed.
To enable GPU acceleration, install CuPy for your CUDA version:
  pip install cupy-cuda12x      # for CUDA 12.x
  pip install cupy-cuda11x     # for CUDA 11.x
See https://docs.cupy.dev/en/stable/install.html

```

```
Do you want to continue in CPU mode? [y/N]:
```

On **macOS** (for Apple Silicon):

```
△ PyTorch not installed (or no Metal support) – running on CPU only.  
To enable Apple Metal GPU acceleration, install PyTorch:  
pip install torch
```

Dependencies (auto-installed):

- Python ≥ 3.9
- NumPy ≥ 1.22
- SciPy ≥ 1.8
- Pandas ≥ 1.5
- Openpyxl ≥ 3.0 (for Excel support)
- Astropy ≥ 5.0 (for FITS support)
- Matplotlib ≥ 3.5
- *(optional)* CuPy — for NVIDIA GPU acceleration
- *(optional)* PyTorch — for Apple Silicon GPU acceleration

Quick Start

Command Line

```
# Analyse a single file (any filename works)  
facg my_lightcurve.dat  
  
# Analyse multiple files in one run  
facg file1.dat file2.dat file3.dat  
  
# Custom parameters  
facg data.dat --sig-limit 6 --oversampling 40 --freq-high 50 --output-dir  
./results  
  
# Quiet mode (suppress progress output)  
facg data.dat -q  
  
# Force CPU-only mode  
facg data.dat --cpu  
  
# Show all available options  
facg --help
```

Python API

```
from facg import FACGConfig, run_analysis

cfg = FACGConfig(
    input_file="data.dat",
    sig_limit=5.0,
    oversampling=20.0,
    freq_high=50.0,      # Upper frequency limit (d-1)
    output_dir="./output", # Output directory
)
results = run_analysis(cfg)

for r in results:
    print(f"  freq = {r['freq']:12.9f}  "
          f"sig = {r['sig']:8.2f}  "
          f"amp = {r['amp']:12.9f}  "
          f"phase = {r['phase']:8.4f}  "
          f"rms = {r['rms']:12.9f}  "
          f"csig = {r['csig']:8.2f}")
```

Input Format

FACG reads **any whitespace-delimited text file** without requiring a specific naming convention or header.

Requirement	Description
Delimiter	Whitespace (spaces or tabs)
Header	None (no header row expected)
Columns	At least 2 numeric columns
Column 0	Time stamps (default, configurable via <code>--time-col</code>)
Column 1	Observed values / flux (default, configurable via <code>--data-col</code>)
Filename	Any valid filename — no rigid naming rules

Example:

```
0.00000000 1.0022292321
0.02097133 1.0026197342
0.03765824 0.9967974816
0.06138463 0.9996638547
...
```

Output Files

All output is written to an automatically created directory `<input_stem>/` next to the input file (or to `--output-dir` if specified).

Main Result Table — `<stem>.dat`

One row per detected frequency signal. Columns:

Column	Description
<code>freq</code>	Frequency (cycles per time unit, e.g. d^{-1})
<code>sig</code>	SigSpec significance (false-alarm probability in $\log_{10}$)
<code>amplitude</code>	Semi-amplitude of the sinusoidal component
<code>phase</code>	Phase angle (radians)
<code>rms</code>	Residual RMS after subtracting this and all prior signals
<code>csig</code>	Cumulative significance of all detections so far

Intermediate Outputs

File	Description	Control flag
<code><stem>_spectrum_NNNN.dat</code>	Significance / amplitude spectrum at iteration N	<code>--no-spectrum</code> to disable
<code><stem>_residuals_NNNN.dat</code>	Residual time series at iteration N	<code>--no-residuals</code> to disable
<code><stem>_spectrum_final.dat</code>	Final residual spectrum	always written
<code><stem>_residuals_final.dat</code>	Final residual time series	always written
<code>phase_NNNN_f*.dat</code>	Folded phase diagram for frequency f	<code>--phase-diagrams</code> to enable

Spectrum files contain 4 columns: `freq`, `sig`, `amplitude`, `phase`.

Residual files contain 2 columns: `time`, `residual`.

Configuration Parameters

All parameters can be set via CLI flags or the `FACGConfig` dataclass.

Frequency Grid

Parameter	CLI Flag	Default	Description
<code>freq_low</code>	<code>--freq-low</code>	Rayleigh resolution $1/T$	Lower frequency limit

Parameter	CLI Flag	Default	Description
<code>freq_high</code>	<code>--freq-high</code>	Nyquist frequency $0.5/\Delta t$	Upper frequency limit
<code>nyquist_coeff</code>	<code>--nyquist-coeff</code>	0.5	Nyquist coefficient
<code>oversampling</code>	<code>--oversampling</code>	20.0	Oversampling ratio (same as SigSpec default)

The frequency grid is constructed as:

```
freq_step = (1/T) / oversampling
grid = [freq_low, freq_low + freq_step, ..., freq_high]
```

Stopping Criteria

Parameter	CLI Flag	Default	Description
<code>sig_limit</code>	<code>--sig-limit</code>	5.0	Stop when peak significance drops below this
<code>csig_limit</code>	<code>--csig-limit</code>	0 (disabled)	Stop when cumulative significance drops below this
<code>max_iter</code>	<code>--max-iter</code>	999	Maximum number of prewhitening iterations

I/O Control

Parameter	CLI Flag	Default	Description
<code>time_col</code>	<code>--time-col</code>	0	0-based column index for time
<code>data_col</code>	<code>--data-col</code>	1	0-based column index for data
<code>output_dir</code>	<code>-o, --output-dir</code>	<code><stem>/</code>	Output directory
<code>write_spectrum</code>	<code>--no-spectrum</code>	True	Write intermediate spectra
<code>write_residuals</code>	<code>--no-residuals</code>	True	Write intermediate residuals
<code>write_phase_diagram</code>	<code>--phase-diagrams</code>	False	Write folded phase diagrams
<code>plot</code>	<code>--plot</code>	False	Generate summary plots (PNG). Disabled by default to save time
<code>quiet</code>	<code>-q, --quiet</code>	False	Suppress progress output

GPU Acceleration Strategy

FACG accelerates two computationally intensive operations using CUDA GPUs via CuPy. When no GPU is available, all computations fall back transparently to NumPy on the CPU.

1. Significance Spectrum — Batch DFT (Primary Acceleration)

The SigSpec significance is evaluated at every frequency in the grid via a direct DFT (not FFT, to handle irregular time sampling):

$$\begin{aligned} a(f) &= (1/N) \sum_i x(t_i) \cdot \cos(2\pi f t_i) \\ b(f) &= (1/N) \sum_i x(t_i) \cdot \sin(2\pi f t_i) \end{aligned}$$

For M frequencies $\times$ N data points, this is formulated as a $(M \times N)$ matrix-vector product computed entirely on the GPU in a single pass (with chunking to limit GPU memory usage).

The significance is then computed analytically from the sampling profile axes α_0 , β_0 and tilt angle θ_0 (Reegen 2007):

$$\text{sig}(f) = N \cdot \log_{10}(e) / \sigma^2 \cdot [(a \cdot \cos\theta_0 + b \cdot \sin\theta_0)^2 / \alpha_0 + (a \cdot \sin\theta_0 - b \cdot \cos\theta_0)^2 / \beta_0]$$

Typical speedup: 10–50× over CPU for $N=5000$, $M=100000$.

2. Multi-Sine Global Optimiser — GPU Jacobian

After each prewhitening step, all detected signals are simultaneously re-fitted to the original data via Levenberg–Marquardt (SciPy `least_squares`). The model is:

$$y(t) = \sum_k A_k \cdot \sin(2\pi f_k t + \phi_k)$$

The Jacobian matrix has analytic partial derivatives:

$$\begin{aligned} \partial y / \partial f_k &= 2\pi t \cdot A_k \cdot \cos(2\pi f_k t + \phi_k) \\ \partial y / \partial A_k &= \sin(2\pi f_k t + \phi_k) \\ \partial y / \partial \phi_k &= A_k \cdot \cos(2\pi f_k t + \phi_k) \end{aligned}$$

The full $(N \times 3K)$ Jacobian is computed on the GPU in one shot, then transferred to the CPU for the LM solver. This avoids SciPy's numerical finite-difference Jacobian approximation (which would require $6K+1$ model evaluations per iteration).

3. Frequency Refinement — Optimised CPU

The bisection frequency refinement (equivalent to SigSpec's `SigSpec_MaxSig`) evaluates significance at ~80 individual frequencies. These are too small for GPU transfer overhead to pay off, so a dedicated

`_sig_single()` function uses `np.dot()` for maximum CPU throughput.

Algorithm Pipeline

The core analysis follows the same iterative prewhitening approach as SigSpec:

```

1. Load & zero-mean the time series
2. Construct frequency grid

  ┌── Prewhitening Cascade Loop ──┐
  │
  │ 3. Compute significance spectrum (GPU)
  │ 4. Find peak significance
  │ 5. Refine frequency (bisection)
  │ 6. Global multi-sine optimise (GPU)
  │ 7. Update residual
  │ 8. Write intermediate outputs
  │
  │ Stop if: sig < sig_limit
  │          or: iter >= max_iter
  │          or: csig < csig_limit
  │
  │ 9. Write final result table & residuals
  │ 10. Report total elapsed time
  │
  └────────────────────────────────┘

```

Comparison with SigSpec

Feature	SigSpec (C)	FACG (Python)
Language	C	Python 3
GPU acceleration	No	Yes (CuPy/CUDA)
Significance formula	Reegen 2007	Same
Iterative prewhitening	Yes	Yes
Global multi-sine fit	Yes	Yes (LM + analytic Jacobian)
Frequency refinement	Bisection (<code>SigSpec_MaxSig</code>)	Same algorithm
Cumulative significance	<code>SigSpec_CSig</code>	Same formula
File naming	Rigid <code><project>/<type><iter>.dat</code>	Flexible, any filename
Installation	Manual compilation	<code>pip install -e .</code>
Multi-file support	Via MultiFile .ini	CLI: <code>facg *.dat</code>

Feature	SigSpec (C)	FACG (Python)
Configuration	<code>.ini</code> file	CLI flags + Python API
Time reporting	No	Per-iteration + total

Example Output

```

=====
FACG – Frequency Analysis of CPU and GPU mixed computing
Backend : CPU (NumPy 2.3.4)
Input   : SigSpec_Test_01.dat
Output  : SigSpec_Test_01
=====
Data points   : 5000
Time base     : 99.999198
Rayleigh res  : 0.010000080
Freq step     : 0.000500004
Freq range    : [0.010000, 50.000000]
Nyquist coeff : 24.980378
Oversampling  : 20.0
# frequencies : 99981
sig threshold : 5.0
=====
iter   1: freq  34.377272957  sig  374.9393  amp  0.002558265  rms
0.002485681  [9.34s]
iter   2: freq  22.945686587  sig  450.3668  amp  0.002274437  rms
0.001901435  [9.40s]
iter   3: freq  31.915168800  sig  580.3440  amp  0.001965941  rms
0.001296536  [9.59s]
iter   4: freq  32.991913360  sig  626.2616  amp  0.001388495  rms
0.000843077  [9.51s]
iter   5: freq  10.329176329  sig  704.0327  amp  0.000962925  rms
0.000499334  [9.52s]
iter   6: max sig = 3.9187 < 5.0 – stopping.
=====
Detected frequencies : 5
Total elapsed time   : 66.422 s
=====

```

Author

Niu Hubiao

License

MIT